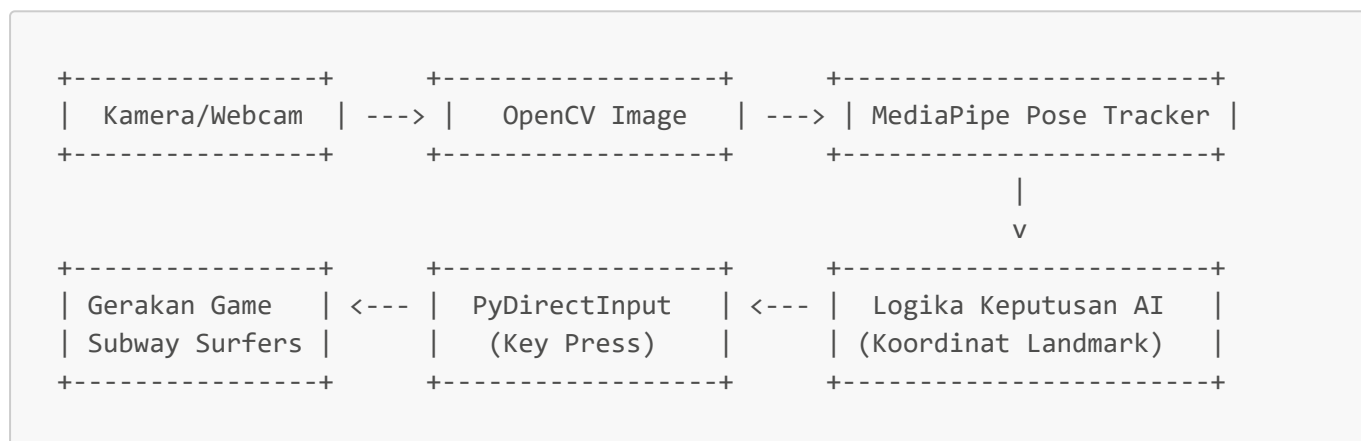


Panduan Penjelasan Kodingan: Subway Surfers AI Controller

Dokumen ini menjelaskan secara menyeluruh cara kerja sistem pelacakan gerakan (Motion Tracker) menggunakan OpenCV & MediaPipe untuk mengendalikan game Subway Surfers di emulator, serta memberikan penjelasan baris demi baris dari file kodingan utama `main.py`.

1. Aliran Flow: Bagaimana Program Menggerakkan Game Subway Surfers?

Sebelum masuk ke penjelasan kode, berikut adalah diagram dan penjelasan alur bagaimana gerakan fisik Anda dapat diterjemahkan menjadi gerakan karakter di dalam emulator **LDPlayer** (Subway Surfers):



Penjelasan Langkah-Langkah:

- Input Kamera (Webcam):** Kamera mengambil gambar/frame Anda secara *real-time* (biasanya 30 frame per detik).
- Pengolahan Gambar (OpenCV):** Gambar dari webcam diubah orientasinya (di-flip/mirror) agar gerakan di layar sesuai dengan gerakan asli Anda (seperti bercermin). Format warna juga diubah dari BGR (standar OpenCV) ke RGB agar siap dibaca oleh pustaka kecerdasan buatan Google MediaPipe.
- Deteksi Sendi Tubuh (MediaPipe):** MediaPipe mendeteksi 33 titik sendi tubuh utama (*Pose Landmarks*). Titik yang kita gunakan adalah:
 - Hidung (Nose):** Untuk mendeteksi posisi geser kanan/kiri.
 - Bahu (Shoulder):** Untuk menghitung ketinggian rata-rata bahu sebagai acuan melompat dan merunduk.
 - Pergelangan Tangan (Wrist):** Untuk mendeteksi tangan yang diangkat ke atas untuk melompat.
- Logika Keputusan AI:** Program membandingkan koordinat titik-titik tersebut dengan garis batas (*threshold*):
 - Jika pergelangan tangan melewati tinggi bahu rata-rata, ini diartikan sebagai **Lompat**.
 - Jika tinggi bahu rata-rata turun melewati garis ungu (*sensitivitas merunduk*), ini diartikan sebagai **Merunduk**.
 - Jika titik hidung bergeser ke kiri dari batas kuning kiri, ini diartikan sebagai **Geser Kiri**.
 - Jika titik hidung bergeser ke kanan dari batas kuning kanan, ini diartikan sebagai **Geser Kanan**.
- Mengirim Sinyal ke Game (PyDirectInput):**

- Game di dalam emulator PC seperti LDPlayer tidak bisa digerakkan menggunakan simulator keyboard biasa (seperti bawaan Windows) karena game modern membaca input keyboard tingkat rendah menggunakan API DirectX (*DirectInput*).
- Pustaka **PyDirectInput** mensimulasikan sinyal keyboard tingkat rendah (*Scancodes*) langsung ke sistem operasi Windows, yang disangka oleh emulator sebagai keyboard fisik sungguhan yang ditekan (tombol Panah Atas, Bawah, Kiri, dan Kanan).
- Akibatnya, karakter di game Subway Surfers akan bergerak secara instan dan responsif sesuai gerakan tubuh Anda.

2. Penjelasan Kodingan `main.py` Baris demi Baris

Berikut adalah penjelasan fungsi setiap baris kode di file [main.py]
(file:///C:/MARSYA%20LASARA/KSC/Project%20Computer%20Vision/main.py):

Bagian 1: Impor Pustaka (Baris 1 - 11)

```
1: import cv2
```

- **Fungsi:** Mengimpor pustaka OpenCV. Digunakan untuk mengakses kamera webcam, membaca frame video, memutar arah video (*flip*), menggambar garis batas koordinat, dan menulis teks bantuan pada video.

```
2: import mediapipe as mp
```

- **Fungsi:** Mengimpor pustaka MediaPipe buatan Google untuk pemrosesan AI pendeteksian kerangka tubuh (*Pose Estimation*).

```
3: import pydirectinput
```

- **Fungsi:** Mengimpor pustaka simulasi tombol keyboard tingkat rendah (*DirectInput*). Ini adalah kunci utama agar tombol panah yang ditekan dapat terdeteksi oleh emulator Android LDPlayer.

```
4: import time
```

- **Fungsi:** Mengimpor pustaka waktu standar Python. Digunakan untuk memberikan waktu tunggu penekanan tombol (*cooldown*) agar tombol tidak tertekan berulang kali dalam milidetik yang sama.

```
5: import csv
```

- **Fungsi:** Mengimpor pustaka CSV. Digunakan untuk menulis data log koordinat tubuh ke dalam format file tabel `.csv` (untuk kebutuhan dataset).

```
6: import os
```

- **Fungsi:** Mengimpor modul OS. Digunakan untuk memanipulasi file sistem, seperti memeriksa apakah file dataset CSV sudah ada atau belum.

```
7: import threading
```

- **Fungsi:** Mengimpor modul threading. Digunakan untuk menjalankan tugas pelacakan kamera di thread latar belakang (*background thread*), sehingga antarmuka grafis (GUI) tidak membeku/lag.

```
8: import tkinter as tk
9: from tkinter import messagebox
```

- **Fungsi:** Mengimpor basis GUI bawaan Python (Tkinter) dan modul dialog kotak pesan (*messagebox*) untuk menampilkan notifikasi error jika kamera tidak ditemukan.

```
10: import customtkinter as ctk
```

- **Fungsi:** Mengimpor CustomTkinter. Ini adalah pustaka pembungkus Tkinter yang memberikan tampilan antarmuka yang sangat modern, mendukung Dark Mode bawaan, pojok melengkung, dan tombol yang estetik.

```
11: from PIL import Image, ImageTk, ImageDraw, ImageFont
```

- **Fungsi:** Mengimpor pustaka manipulasi gambar (Pillow). Digunakan untuk mengonversi gambar video OpenCV (numpy array) menjadi format yang bisa ditampilkan oleh Tkinter, serta menggambar gambar placeholder saat kamera dinonaktifkan.

Bagian 2: Konfigurasi Tema Awal (Baris 13 - 15)

```
14: ctk.set_appearance_mode("dark")
```

- **Fungsi:** Mengatur tema tampilan aplikasi ke mode gelap (*Dark Mode*).

```
15: ctk.set_default_color_theme("blue")
```

- **Fungsi:** Mengatur tema warna aksent utama widget (tombol, slider) ke warna biru.

Bagian 3: Konstruktor Kelas Utama (Baris 17 - 58)

```
17: class SubwaySurfersAIApp(ctk.CTk):
```

- **Fungsi:** Mendefinisikan kelas aplikasi utama kita yang mewarisi sifat dari jendela utama CustomTkinter (`ctk.CTk`).

```
18:     def __init__(self):  
19:         super().__init__()
```

- **Fungsi:** Fungsi inisialisasi awal (*constructor*) kelas. Baris 19 memanggil konstruktor bawaan dari kelas induk `ctk.CTk`.

```
22:         self.title("AI Subway Surfers Controller & Motion Tracker")
```

- **Fungsi:** Menentukan judul jendela aplikasi yang akan muncul di pojok kiri atas layar.

```
23:         self.width = 1020  
24:         self.height = 560  
25:         self.center_window(self.width, self.height)
```

- **Fungsi:** Menentukan dimensi lebar (1020 piksel) dan tinggi (560 piksel) jendela, lalu memanggil fungsi buatan sendiri `center_window` agar aplikasi muncul tepat di tengah layar monitor pengguna saat dibuka.

```
26:         self.resizable(False, False)
```

- **Fungsi:** Mengunci ukuran jendela aplikasi agar tidak bisa diperkecil atau diperbesar oleh pengguna (menghindari visual layout berantakan).

```
29:         self.lock = threading.Lock()
```

- **Fungsi:** Membuat objek Thread Lock. Ini sangat penting untuk menjaga integritas data saat thread GUI (thread utama) dan thread OpenCV (thread latar belakang) membaca/menulis variabel yang sama agar tidak terjadi tabrakan memori (*race condition*).

```
30:         self.is_running = False
```

- **Fungsi:** Bendera (*flag*) boolean penanda apakah sistem deteksi kamera sedang berjalan (**True**) atau berhenti (**False**).

```
31:         self.latest_frame = None
```

- **Fungsi:** Variabel penampung frame video terbaru yang ditangkap dari webcam untuk dikirim dari thread kamera ke thread GUI.

```
32:         self.current_position = "Center"
```

- **Fungsi:** Menyimpan posisi sumbu horizontal hidung user secara real-time (**Left**, **Center**, atau **Right**).

```
33:         self.status_sekarang = "Normal"
```

- **Fungsi:** Menyimpan aksi/gerakan aktif yang sedang dikirim ke game (**Lompat**, **Merunduk**, **Geser Kiri**, **Geser Kanan**, atau **Normal**).

```
35:         self.nama_file_dataset = "dataset_koordinat_subway.csv"
```

- **Fungsi:** Menentukan nama file tempat penyimpanan dataset koordinat gerakan.

```
36:         self.log_count = self.get_csv_rows(self.nama_file_dataset)
```

- **Fungsi:** Menghitung jumlah baris data koordinat yang sudah ada di file CSV saat aplikasi pertama kali dibuka.

```
37:         self.persentase_merunduk = 0.53
```

- **Fungsi:** Menentukan nilai awal batas horizontal untuk gerakan merunduk (53% dari tinggi layar).

```
40:         self.grid_rowconfigure(0, weight=1)
41:         self.grid_columnconfigure(0, weight=0)
42:         self.grid_columnconfigure(1, weight=1)
```

- **Fungsi:** Mengatur sistem tata letak kisi (*grid layout*) jendela utama. Kolom 0 (Sidebar) memiliki lebar tetap, sedangkan Kolom 1 (Tampilan Kamera) akan menyesuaikan dengan sisa ruang yang ada.

```
45:         self.sidebar_frame = ctk.CTkFrame(self, width=320, corner_radius=15,
fg_color="#18191D")
46:         self.sidebar_frame.grid(row=0, column=0, padx=20, pady=20,
sticky="nsew")
47:         self.sidebar_frame.grid_propagate(False)
```

- **Fungsi:** Membuat panel Sidebar di sisi kiri dengan warna latar abu-abu gelap, pojok melengkung 15 piksel, lebar 320 piksel, dan mematikan propagasi agar ukurannya tidak mengecil secara otomatis jika isi widget di dalamnya berubah.

```
50:         self.camera_frame = ctk.CTkFrame(self, width=640, height=480,
corner_radius=15, fg_color="#18191D")
51:         self.camera_frame.grid(row=0, column=1, padx=(0, 20), pady=20,
sticky="nsew")
```

- **Fungsi:** Membuat panel wadah untuk video kamera di sebelah kanan dengan ukuran 640x480 piksel.

```
53:         self.setup_sidebar()
54:         self.setup_camera_view()
```

- **Fungsi:** Memanggil fungsi pembuatan elemen detail visual sidebar dan tampilan kamera.

```
57:         self.protocol("WM_DELETE_WINDOW", self.on_closing)
```

- **Fungsi:** Mendaftarkan protokol jika pengguna menekan tombol tutup window **X**. Ini memicu fungsi `on_closing` agar program menghentikan kamera dan thread terlebih dahulu secara bersih sebelum keluar.

Bagian 4: Fungsi Pembantu (Baris 59 - 73)

```
59:     def center_window(self, width, height):
60:         screen_width = self.winfo_screenwidth()
61:         screen_height = self.winfo_screenheight()
```

```

62:         x = (screen_width - width) // 2
63:         y = (screen_height - height) // 2
64:         self.geometry(f"{width}x{height}+{x}+{y}")

```

- **Fungsi:** Mengambil data resolusi lebar dan tinggi layar monitor fisik pengguna, lalu menghitung koordinat X dan Y agar jendela aplikasi diposisikan tepat di tengah monitor.

```

66:     def get_csv_rows(self, filename):
67:         if not os.path.exists(filename):
68:             return 0
69:         try:
70:             with open(filename, mode='r') as f:
71:                 return sum(1 for line in f) - 1
72:         except Exception:
73:             return 0

```

- **Fungsi:** Membuka file dataset CSV secara aman, menghitung total baris di dalamnya, dan mengurangi 1 (karena baris pertama adalah judul kolom). Jika terjadi error atau file tidak ada, mengembalikan nilai 0.

Bagian 5: Penyusunan Desain Sidebar (Baris 75 - 178)

```

77:         self.title_label = ctk.CTkLabel(
78:             self.sidebar_frame,
79:             text="SUBWAY SURFERS",
80:             font=ctk.CTkFont(family="Inter", size=24, weight="bold"),
81:             text_color="#3498db"
82:         )
83:         self.title_label.pack(pady=(25, 0), padx=20, anchor="w")

```

- **Fungsi:** Membuat teks judul besar "SUBWAY SURFERS" berwarna biru dengan gaya font Inter Tebal berukuran 24 di pojok kiri atas sidebar.

```

85:         self.subtitle_label = ctk.CTkLabel(...)
...
91:         self.subtitle_label.pack(pady=(2, 20), padx=20, anchor="w")

```

- **Fungsi:** Menambahkan teks sub-judul kecil berwarna abu-abu muda langsung di bawah judul utama.

```

94:         self.separator = ctk.CTkFrame(self.sidebar_frame, height=2,
fg_color="#2c2c2e")
95:         self.separator.pack(fill="x", padx=20, pady=(0, 20))

```

- **Fungsi:** Menggambar garis pemisah horizontal tipis berwarna abu-abu gelap agar UI terlihat lebih rapi dan terorganisir.

```
98:         self.btn_control = ctk.CTkButton(
99:             self.sidebar_frame,
100:             text="MULAI AI TRACKER",
101:             ...
106:             corner_radius=10,
107:             command=self.toggle_tracker
108:         )
109:         self.btn_control.pack(fill="x", padx=20, pady=(0, 20))
```

- **Fungsi:** Membuat tombol kontrol utama bertuliskan "MULAI AI TRACKER" dengan latar hijau cerah. Tombol ini memiliki sudut lengkung 10 piksel dan akan memicu fungsi `toggle_tracker` saat diklik.

```
112:         self.settings_frame = ctk.CTkFrame(self.sidebar_frame,
113:             fg_color="#202126", corner_radius=10)
114:         self.settings_frame.pack(fill="x", padx=20, pady=(0, 20))
```

- **Fungsi:** Membuat kotak wadah abu-abu gelap khusus untuk elemen pengaturan sensitivitas.

```
115:         self.slider_title = ctk.CTkLabel(
116:             self.settings_frame,
117:             text=f"Sensitivitas Merunduk: {self.persentase_merunduk:.2f}",
118:             ...
121:         self.slider_title.pack(pady=(12, 4), padx=15, anchor="w")
```

- **Fungsi:** Label teks penunjuk nilai sensitivitas merunduk yang aktif. Teks ini akan ter-update setiap kali slider digeser.

```
123:         self.slider_duck = ctk.CTkSlider(
124:             self.settings_frame,
125:             from_=0.3,
126:             to=0.7,
127:             number_of_steps=40,
128:             command=self.on_slider_move
129:         )
130:         self.slider_duck.set(self.persentase_merunduk)
131:         self.slider_duck.pack(fill="x", padx=15, pady=(0, 12))
```

- **Fungsi:** Membuat widget slider geser dengan rentang nilai dari 0.3 hingga 0.7 (dibagi menjadi 40 anak tangga) untuk mengubah variabel batas merunduk saat program berjalan melalui fungsi `on_slider_move`.

```
134:         self.status_frame = ctk.CTkFrame(self.sidebar_frame,
fg_color="#202126", corner_radius=10)
135:         self.status_frame.pack(fill="both", expand=True, padx=20, pady=(0,
25))
```

- **Fungsi:** Membuat kotak wadah monitor status di bagian paling bawah sidebar.

```
138:         self.status_header = ctk.CTkLabel(..., text="MONITOR STATUS", ...)
```

- **Fungsi:** Membuat judul bagian monitor status.

```
146:         self.lbl_status = self.create_status_row(self.status_frame, "Status
Pelacak:", "STANDBY")
147:         self.val_status = self.lbl_status[1]
148:         self.val_status.configure(text_color="gray")
```

- **Fungsi:** Membuat baris status pelacak kamera dengan nilai awal "STANDBY" berwarna abu-abu.

```
150:         self.lbl_position = self.create_status_row(self.status_frame, "Posisi
Sumbu:", "-")
151:         self.val_position = self.lbl_position[1]
```

- **Fungsi:** Membuat baris monitor posisi horizontal tubuh.

```
153:         self.lbl_gesture = self.create_status_row(self.status_frame, "Gerakan
Aktif:", "-")
154:         self.val_gesture = self.lbl_gesture[1]
```

- **Fungsi:** Membuat baris monitor gerakan keyboard yang sedang dikirim ke game.

```
156:         self.lbl_csv = self.create_status_row(self.status_frame, "Log CSV:",
f"{self.log_count} baris")
157:         self.val_csv = self.lbl_csv[1]
```

- **Fungsi:** Membuat baris indikator jumlah data CSV terdaftar.

```
159:     def create_status_row(self, parent, label_text, val_text):  
...  
178:         return lbl, val
```

- **Fungsi:** Fungsi pembantu berulang untuk membuat baris informasi (kiri berisikan nama label abu-abu, kanan berisikan nilai status tebal putih).

Bagian 6: Area Tampilan Video & Placeholder (Baris 180 - 214)

```
180:     def setup_camera_view(self):  
181:         self.camera_label = tk.Label(self.camera_frame, bg="#111215")  
182:         self.camera_label.pack(fill="both", expand=True, padx=2, pady=2)  
183:         self.show_placeholder()
```

- **Fungsi:** Menambahkan label visual dasar Tkinter (`tk.Label`) dengan latar belakang sangat gelap ke dalam panel kamera kanan untuk merender video webcam, lalu memanggil fungsi placeholder kamera nonaktif.

```
186:     def show_placeholder(self):  
187:         placeholder = Image.new("RGB", (640, 480), color="#111215")  
188:         draw = ImageDraw.Draw(placeholder)
```

- **Fungsi:** Membuat gambar kosong beresolusi 640x480 piksel berwarna hitam keabuan dan membuat objek kanvas gambar `draw` untuk mencoret-coret visual.

```
194:         draw.ellipse((270, 160, 370, 260), outline="#2a2b30", width=4)  
195:         draw.line((295, 210, 345, 210), fill="#2a2b30", width=6)  
196:         draw.line((320, 185, 320, 235), fill="#2a2b30", width=6)
```

- **Fungsi:** Menggambar ilustrasi logo lingkaran kamera sederhana lengkap dengan simbol plus di tengahnya menggunakan modul Pillow Draw sebagai estetika UI.

```
205:         draw.text((320, 290), text1, fill="ffffff", anchor="mm", font=font1)  
206:         draw.text((320, 330), text2, fill="#7f8c8d", anchor="mm", font=font2)
```

- **Fungsi:** Menggambar teks instruksi "Kamera Dinonaktifkan" dan cara menyalakan program secara presisi di tengah layar (menggunakan properti `anchor="mm"`).

```
208:         img_tk = ImageTk.PhotoImage(image=placeholder)  
209:         self.camera_label.configure(image=img_tk)
```

```
210:         self.camera_label.image = img_tk
```

- **Fungsi:** Mengonversi objek gambar Pillow tersebut ke format yang kompatibel dengan Tkinter (`PhotoImage`), lalu menampilkannya langsung ke label layar kamera.

```
212:     def on_slider_move(self, val):
213:         self.persentase_merunduk = float(val)
214:         self.slider_title.configure(text=f"Sensitivitas Merunduk:
{self.persentase_merunduk:.2f}")
```

- **Fungsi:** Fungsi callback yang dieksekusi secara instan setiap kali slider digeser. Ini meng-update variabel sensitivitas merunduk global dan memperbarui teks label slider di atasnya.

Bagian 7: Sakelar Mulai/Berhenti & Thread Utama (Baris 216 - 286)

```
216:     def toggle_tracker(self):
```

- **Fungsi:** Fungsi sakelar ketika tombol kontrol diklik.

```
217:         if not self.is_running:
```

- **Fungsi:** Jika program dalam kondisi mati (belum berjalan):

```
219:             if hasattr(self, 'thread') and self.thread.is_alive():
220:                 return
```

- **Fungsi:** Cek apakah thread sebelumnya masih aktif melepaskan memori. Jika iya, batalkan proses untuk mencegah tumpang tindih.

```
223:             self.is_running = True
224:             self.btn_control.configure(
225:                 text="BERHENTI AI TRACKER",
226:                 fg_color="#e74c3c",
227:                 hover_color="#c0392b"
228:             )
```

- **Fungsi:** Mengubah bendera berjalan ke `True`, mengubah teks tombol menjadi "BERHENTI AI TRACKER", dan mengubah warna tombol ke warna merah/merah bata gelap.

```
231:         self.thread = threading.Thread(target=self.worker_loop,  
daemon=True)  
232:         self.thread.start()
```

- **Fungsi:** Membuat thread latar belakang baru untuk mengeksekusi fungsi `worker_loop` (logika webcam & AI) agar terpisah dari thread antarmuka, dan menandainya sebagai `daemon` agar mati otomatis jika aplikasi utama ditutup. `thread.start()` mulai menjalankan fungsinya.

```
235:         self.after(15, self.update_gui)
```

- **Fungsi:** Memulai loop waktu internal Tkinter. Fungsi ini memerintahkan GUI untuk mengeksekusi metode `update_gui` setelah 15 milidetik.

```
236:         else:  
237:             self.is_running = False  
238:             self.btn_control.configure(  
239:                 text="MULAI AI TRACKER",  
240:                 fg_color="#2ecc71",  
241:                 hover_color="#27ae60"  
242:             )
```

- **Fungsi:** Jika program dalam kondisi berjalan dan diklik, matikan pelacakan (`is_running = False`) dan kembalikan desain tombol ke semula (hijau bertuliskan "MULAI AI TRACKER").

```
246:     def update_gui(self):
```

- **Fungsi:** Fungsi loop pembaruan antarmuka utama (berjalan setiap 15ms).

```
247:         if self.is_running:
```

- **Fungsi:** Jika program sedang aktif mendeteksi:

```
248:             with self.lock:  
249:                 frame = self.latest_frame  
250:                 position = self.current_position  
251:                 status = self.status_sekarang  
252:                 logs = self.log_count
```

- **Fungsi:** Masuk ke blok proteksi `lock` untuk mengambil data frame video terbaru beserta variabel koordinat dari background thread dengan aman tanpa risiko tabrakan memori.

```

254:         if frame is not None:
255:             img = Image.fromarray(frame)
256:             img_tk = ImageTk.PhotoImage(image=img)
257:             self.camera_label.configure(image=img_tk)
258:             self.camera_label.image = img_tk

```

- **Fungsi:** Jika ada frame baru yang diterima, ubah frame OpenCV (numpy array RGB) tersebut menjadi objek Gambar Pillow, lalu ubah ke format Tkinter PhotoImage dan tampilkan di layar panel kanan.

```

260:         self.val_status.configure(text="AKTIF", text_color="#2ecc71")
261:         self.val_position.configure(
262:             text=position,
263:             text_color="#3498db" if position == "Center" else "#f1c40f"
264:         )

```

- **Fungsi:** Mengubah teks monitor status menjadi "AKTIF" berwarna hijau dan memperbarui posisi sumbu tubuh (jika berada di tengah berwarna biru, jika condong ke kiri/kanan berwarna kuning).

```

266:         # Dynamic colors for active gestures
267:         gesture_color = "#ffffff"
268:         if status == "Lompat":
269:             gesture_color = "#2ecc71"
270:         elif status == "Merunduk":
271:             gesture_color = "#e74c3c"
272:         elif status in ["Geser Kiri", "Geser Kanan"]:
273:             gesture_color = "#f39c12"
274:
275:         self.val_gesture.configure(text=status, text_color=gesture_color)

```

- **Fungsi:** Menentukan warna teks yang dinamis dan kontras berdasarkan gerakan tubuh aktif saat ini (Lompat = hijau, Merunduk = merah, Geser = oranye).

```

276:         self.val_csv.configure(text=f"{logs} baris",
text_color="#ffffff")

```

- **Fungsi:** Memperbarui counter jumlah baris log CSV yang terdaftar.

```

278:         self.after(15, self.update_gui)

```

- **Fungsi:** Menjadwalkan kembali pemanggilan fungsi `update_gui` dalam 15ms ke depan (sehingga membentuk loop rekursif waktu internal).

```
279:         else:
280:             self.val_status.configure(text="STANDBY", text_color="gray")
281:             self.val_position.configure(text="-", text_color="gray")
282:             self.val_gesture.configure(text="-", text_color="gray")
283:             with self.lock:
284:                 logs = self.log_count
285:             self.val_csv.configure(text=f"{logs} baris", text_color="gray")
286:             self.show_placeholder()
```

- **Fungsi:** Jika program dinonaktifkan (`is_running = False`), atur ulang semua tampilan monitor status sidebar kembali ke tanda strip (-) berwarna abu-abu dan kembalikan layar video kanan ke gambar placeholder.

Bagian 8: Logika Thread Kamera & AI Pelacakan (Baris 288 - 364)

```
288:     def worker_loop(self):
```

- **Fungsi:** Fungsi utama loop pemrosesan video kamera dan deteksi AI MediaPipe (berjalan di thread terpisah).

```
290:         from mediapipe.python.solutions import pose as mp_pose
291:         from mediapipe.python.solutions import drawing_utils as mp_drawing
```

- **Fungsi:** Melakukan impor lokal MediaPipe di dalam thread untuk menghemat beban inisialisasi awal program utama.

```
294:         pose = mp_pose.Pose(min_detection_confidence=0.5,
min_tracking_confidence=0.5)
```

- **Fungsi:** Menginisialisasi objek model pose MediaPipe dengan batas minimal kepercayaan deteksi 50% dan pelacakan 50%.

```
297:         cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
298:         if not cap.isOpened():
299:             cap = cv2.VideoCapture(0)
```

- **Fungsi:** Membuka webcam pertama di komputer. Coba menggunakan API DirectShow (`cv2.CAP_DSHOW`) khusus Windows agar kamera terbuka lebih cepat. Jika gagal, coba membuka menggunakan metode API standar bawaan sistem.

```

301:         if not cap.isOpened():
302:             # Failed to open camera
303:             with self.lock:
304:                 self.status_sekarang = "Kamera Error"
305:                 self.current_position = "-"
306:                 self.is_running = False
307:                 self.after(100, lambda: messagebox.showerror("Koneksi Kamera
Gagal", "Tidak dapat mendeteksi atau membuka webcam. Harap periksa koneksi kamera
Anda."))
308:                 return

```

- **Fungsi:** Jika kamera gagal diakses (misal karena rusak atau sedang dipakai aplikasi lain), ubah status menjadi eror, matikan bendera program berjalan, lalu tampilkan dialog eror ke layar utama pengguna.

```

311:         cooldown_time = 0.4

```

- **Fungsi:** Waktu jeda antarggerakan penekanan tombol keyboard (0.4 detik) untuk mencegah sistem menekan tombol secara liar berulang kali hanya karena satu gerakan lambat.

```

312:         last_action_time = time.time()

```

- **Fungsi:** Menyimpan data timestamp kapan terakhir kali gerakan keyboard dieksekusi.

```

313:         current_position_local = "Center"

```

- **Fungsi:** Variabel lokal thread penampung posisi horizontal hidung.

```

316:         if not os.path.exists(self.nama_file_dataset):
317:             with open(self.nama_file_dataset, mode='w', newline='') as file:
318:                 writer = csv.writer(file)
319:                 writer.writerow(['Timestamp', 'X_Hidung', 'Y_Bahu_Rata2',
'Posisi_Sumbu', 'Status_Gerakan'])

```

- **Fungsi:** Cek apakah file dataset CSV ada di dalam folder. Jika belum ada, buat file baru dan tulis judul kolom utamanya sebagai baris pertama.

```

321:         while self.is_running:

```

- **Fungsi:** Memulai loop utama membaca frame video selama bendera program berjalan bernilai **True**.

```
322:         ret, frame = cap.read()
323:         if not ret:
324:             cap.release()
325:             cap = cv2.VideoCapture(0)
326:             time.sleep(0.5)
327:             continue
```

- **Fungsi:** Membaca frame terbaru dari webcam. Jika pembacaan frame gagal (misal kabel kamera longgar), coba hubungkan kembali kamera dan berikan jeda waktu 0.5 detik sebelum mengulangi loop.

```
329:         frame = cv2.flip(frame, 1)
```

- **Fungsi:** Membalik gambar video secara horizontal (efek mirror) agar gerakan pengguna sinkron kirikanannya di monitor.

```
330:         height, width, _ = frame.shape
```

- **Fungsi:** Mengambil dimensi resolusi piksel lebar (*width*) dan tinggi (*height*) dari video webcam yang aktif.

```
331:         rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

- **Fungsi:** Mengubah standar format warna video dari BGR (OpenCV) menjadi format warna RGB (MediaPipe).

```
332:         results = pose.process(rgb_frame)
```

- **Fungsi:** Memasukkan gambar RGB ke dalam model AI MediaPipe Pose untuk dideteksi titik-titik kerangka tubuhnya.

```
335:         current_persentase_merunduk = self.persentase_merunduk
```

- **Fungsi:** Mengambil nilai sensitivitas merunduk terbaru yang sedang diatur pengguna dari slider GUI.

```
338:         cv2.line(frame, (int(width * 0.35), 0), (int(width * 0.35),
height), (0, 255, 255), 2)
339:         cv2.line(frame, (int(width * 0.65), 0), (int(width * 0.65),
height), (0, 255, 255), 2)
```

- **Fungsi:** Menggambar garis panduan kuning vertikal di batas 35% lebar layar dan 65% lebar layar sebagai acuan batas geser kiri/kanan.

```
342:         cv2.line(frame, (0, int(height * current_persentase_merunduk)),
(width, int(height * current_persentase_merunduk)), (255, 0, 255), 2)
343:         cv2.putText(frame, "BATAS MERUNDUK", (10, int(height *
current_persentase_merunduk) - 10),
344:                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 255), 1)
```

- **Fungsi:** Menggambar garis batas merunduk horizontal berwarna ungu (magenta) sesuai persentase yang diatur slider dan menuliskan teks keterangannya di atas garis.

```
346:         status_sekarang_local = "Normal"
```

- **Fungsi:** Mengatur status gerakan lokal awal ke "Normal" pada setiap frame pembacaan.

Bagian 9: Pengolahan Logika Gerakan AI & Input Keyboard (Baris 348 - 420)

```
348:         if results.pose_landmarks:
```

- **Fungsi:** Memeriksa apakah ada kerangka tubuh manusia terdeteksi di dalam frame kamera.

```
349:         mp_drawing.draw_landmarks(frame, results.pose_landmarks,
mp_pose.POSE_CONNECTIONS)
```

- **Fungsi:** Menggambar garis-garis kerangka sendi tubuh berwarna merah secara langsung di atas video frame untuk membantu pengguna mengetahui apakah tubuh mereka terlacak dengan baik oleh AI.

```
350:         landmarks = results.pose_landmarks.landmark
```

- **Fungsi:** Mengambil daftar data koordinat 33 titik sendi tubuh.

```
352:         nose_x = int(landmarks[mp_pose.PoseLandmark.NOSE].x * width)
```

- **Fungsi:** Mengambil koordinat posisi hidung secara horizontal (sumbu X) dan mengonversinya dari rasio 0-1 menjadi koordinat piksel layar sebenarnya.

```
353:         left_shoulder_y =  
int(landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER].y * height)  
354:         right_shoulder_y =  
int(landmarks[mp_pose.PoseLandmark.RIGHT_SHOULDER].y * height)  
355:         avg_shoulder_y = (left_shoulder_y + right_shoulder_y) // 2
```

- **Fungsi:** Mengambil posisi tinggi (sumbu Y) bahu kiri dan bahu kanan, lalu menghitung rata-ratanya sebagai garis referensi bahu.

```
358:         cv2.line(frame, (0, avg_shoulder_y), (width, avg_shoulder_y),  
(0, 255, 0), 2)  
359:         cv2.putText(frame, "BATAS LOMPAT (ANGKAT TANGAN)", (width -  
250, avg_shoulder_y - 10),  
360:                     cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 255, 0), 1)
```

- **Fungsi:** Menggambar garis hijau horizontal setinggi bahu rata-rata pengguna saat ini dan menulis teks keterangannya.

```
362:         left_wrist_y =  
int(landmarks[mp_pose.PoseLandmark.LEFT_WRIST].y * height)  
363:         right_wrist_y =  
int(landmarks[mp_pose.PoseLandmark.RIGHT_WRIST].y * height)
```

- **Fungsi:** Mengambil tinggi (sumbu Y) pergelangan tangan kiri dan pergelangan tangan kanan pengguna.

```
366:         current_time = time.time()  
367:         if current_time - last_action_time > cooldown_time:
```

- **Fungsi:** Mengambil waktu saat ini dan memeriksa apakah jeda waktu sejak aksi penekanan tombol terakhir sudah melebihi batas *cooldown* (0.4 detik). Jika iya, program siap membaca gerakan baru.

```
370:         if left_wrist_y < avg_shoulder_y or right_wrist_y <  
avg_shoulder_y:
```

- **Fungsi: Logika Lompat:** Memeriksa apakah pergelangan tangan kiri ATAU pergelangan tangan kanan diangkat hingga posisinya secara koordinat Y berada di atas (nilai piksel Y lebih kecil) dari ketinggian bahu rata-rata.

```

371:                                     try:
372:                                         pydirectinput.keyDown('up')
373:                                         time.sleep(0.05)
374:                                         pydirectinput.keyUp('up')
375:                                     except Exception:
376:                                         pass

```

- **Fungsi:** Menekan tombol keyboard **Panah Atas (Up Arrow)** selama 0.05 detik menggunakan PyDirectInput untuk menginstruksikan karakter Subway Surfers melompat. Blok **try-except** mencegah crash jika terjadi eror sistem input OS.

```

377:                                     cv2.putText(frame, "LOMPAT (PANAHA ATAS)", (50, 60),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 3)
378:                                     status_sekarang_local = "Lompat"
379:                                     last_action_time = current_time

```

- **Fungsi:** Menuliskan teks besar berwarna hijau di layar video penanda tombol lompat terpicu, mengubah status gerakan lokal ke "Lompat", dan memperbarui timestamp aksi terakhir.

```

382:                                     elif avg_shoulder_y > (height *
current_persentase_merunduk):

```

- **Fungsi: Logika Merunduk:** Memeriksa apakah ketinggian bahu rata-rata turun hingga posisinya secara koordinat Y berada di bawah (nilai piksel Y lebih besar) dari persentase garis ungu pembatas merunduk.

```

383:                                     try:
384:                                         pydirectinput.keyDown('down')
385:                                         time.sleep(0.05)
386:                                         pydirectinput.keyUp('down')
...
390:                                     status_sekarang_local = "Merunduk"
391:                                     last_action_time = current_time

```

- **Fungsi:** Menekan tombol keyboard **Panah Bawah (Down Arrow)** selama 0.05 detik untuk membuat karakter merunduk/meluncur, menuliskan info di layar video dengan warna merah, dan memperbarui status gerakan.

```

394:                                     elif nose_x < int(width * 0.35):

```

```
395:             if current_position_local != "Left":
```

- **Fungsi: Logika Geser Kiri:** Memeriksa apakah koordinat X hidung berada di sebelah kiri batas 35% lebar video. Kondisi kedua memastikan gerakan hanya dikirim sekali saat pertama kali masuk ke area kiri (bukan berulang-ulang saat menatap area kiri terus-menerus).

```
396:             try:
397:                 pydirectinput.keyDown('left')
398:                 time.sleep(0.05)
399:                 pydirectinput.keyUp('left')
...
402:             current_position_local = "Left"
403:             status_sekarang_local = "Geser Kiri"
404:             last_action_time = current_time
```

- **Fungsi:** Menekan tombol keyboard **Panah Kiri (Left Arrow)** selama 0.05 detik untuk menggerakkan karakter ke kiri, memperbarui posisi lokal tubuh ke "Left", dan memperbarui status.

```
407:             elif nose_x > int(width * 0.65):
408:                 if current_position_local != "Right":
```

- **Fungsi: Logika Geser Kanan:** Memeriksa apakah koordinat X hidung berada di sebelah kanan batas 65% lebar video, serta menjaga agar tidak mengirim tombol terus-menerus jika posisi tubuh sudah berada di sebelah kanan.

```
409:             try:
410:                 pydirectinput.keyDown('right')
411:                 time.sleep(0.05)
412:                 pydirectinput.keyUp('right')
...
415:             current_position_local = "Right"
416:             status_sekarang_local = "Geser Kanan"
417:             last_action_time = current_time
```

- **Fungsi:** Menekan tombol keyboard **Panah Kanan (Right Arrow)** selama 0.05 detik untuk menggerakkan karakter ke kanan, memperbarui posisi lokal tubuh ke "Right", dan memperbarui status.

```
419:             else:
420:                 current_position_local = "Center"
```

- **Fungsi:** Jika hidung berada di antara batas 35% dan 65%, posisikan sumbu tubuh lokal ke "Center".

Bagian 10: Pencatatan CSV & Pengiriman Frame (Baris 422 - 455)

```
423:                 if status_sekarang_local != "Normal":
```

- **Fungsi:** Jika ada gerakan aktif terdeteksi (Lompat, Merunduk, Geser Kiri, Geser Kanan) dan bukan status Normal:

```
424:                 waktu_log = time.strftime("%Y-%m-%d %H:%M:%S")
425:                 with open(self.nama_file_dataset, mode='a',
newline='') as file:
426:                     writer = csv.writer(file)
427:                     writer.writerow([waktu_log, nose_x,
avg_shoulder_y, current_position_local, status_sekarang_local])
```

- **Fungsi:** Mengambil waktu log saat ini dengan format tanggal dan jam, lalu membuka file CSV untuk menambahkan baris data baru koordinat deteksi sebagai rekaman dataset dosen.

```
428:                 with self.lock:
429:                     self.log_count += 1
```

- **Fungsi:** Masuk ke blok `lock` untuk menambahkan total log dataset sebanyak satu baris agar terlihat perubahannya secara real-time pada UI sidebar.

```
431:             else:
432:                 cv2.putText(frame, "PASTIKAN KAMERA MENYALA & MUNDUR
SEBENTAR", (50, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
```

- **Fungsi:** Jika kerangka tubuh tidak terdeteksi oleh AI MediaPipe, tampilkan teks instruksi peringatan berwarna merah di atas layar video.

```
433:                 cv2.putText(frame, f"Posisi Tubuh: {current_position_local}",
(50, height - 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 0, 0), 2)
```

- **Fungsi:** Menggambar teks posisi horizontal tubuh aktif saat ini di pojok kiri bawah video sebagai umpan balik visual untuk user.

```
436:                 rgb_display_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

- **Fungsi:** Mengonversi video frame akhir dari standard OpenCV (BGR) yang sudah dicoret-coret dengan garis batas ke standard warna RGB agar warnanya tidak salah (misal garis kuning tidak berubah menjadi

biru) saat dirender di GUI Tkinter.

```
438:         with self.lock:
439:             self.latest_frame = rgb_display_frame
440:             self.current_position = current_position_local
441:             self.status_sekarang = status_sekarang_local
```

- **Fungsi:** Masuk ke blok thread `lock` untuk memperbarui variabel global thread (`latest_frame`, `current_position`, dan `status_sekarang`) agar dapat langsung diambil dan ditampilkan oleh thread antarmuka GUI utama.

```
443:         cap.release()
```

- **Fungsi:** Kode ini dipanggil jika loop `while self.is_running` berhenti. Ini melepas webcam secara bersih sehingga kamera tidak terkunci oleh sistem operasi dan lampu indikator kamera mati.

Bagian 11: Penutupan Jendela & Eksekusi Utama (Baris 445 - 455)

```
445:     def on_closing(self):
446:         self.is_running = False
```

- **Fungsi:** Fungsi pembersihan saat tombol tutup aplikasi `X` ditekan. Pertama, bendera berjalan diset ke `False` untuk segera memutus loop di thread kamera.

```
449:         if hasattr(self, 'thread') and self.thread.is_alive():
450:             self.thread.join(timeout=1.0)
```

- **Fungsi:** Menunggu thread latar belakang kamera menyelesaikan pembersihan kamera hingga maksimal 1.0 detik agar program tidak memicu *hanging state* (jendela hilang tapi proses python masih berjalan di Task Manager).

```
451:         self.destroy()
```

- **Fungsi:** Menghancurkan jendela Tkinter dan mematikan program utama sepenuhnya.

```
453: if __name__ == "__main__":
454:     app = SubwaySurfersAIApp()
455:     app.mainloop()
```

- **Fungsi:** Titik masuk utama program (*entry point*). Baris 454 membuat instansiasi objek aplikasi, dan baris 455 menjalankan loop utama pemrosesan GUI Tkinter (`mainloop()`) untuk menangkap klik tombol, pergerakan slider, dan merender visual antarmuka ke layar monitor secara tidak terbatas hingga ditutup.